

Deception CTF Write-up

Author: @elesquina

challenge name: *Deception*

Reverse Engineering / Malware Analysis — 500 pts

Challenge Overview

Our SOC team received a suspicious Windows executable in a password-protected archive (`ZIP password: infected`). The objective was to safely analyze the sample and recover the flag. The binary is not harmful in this CTF context, but analysis was performed as if it were a real incident: offline, isolated, and with minimal execution.

Unpacking the ZIP (AES-encrypted)

The archive couldn't be extracted with a standard `unzip` build due to WinZip AES encryption (PK compat v5.1). Using Python + `pyzipper` solved extraction cleanly:

```
python3 -m pip install pyzipper
python3 - <<'PY'
import pyzipper, os
with pyzipper.AESZipFile("Deception.exe.zip") as z:
    z.pwd = b"infected"
    z.extractall("out")
print("Extracted:", os.listdir("out"))
PY
```

Static Triage

First, I identified the file type and calculated hashes for tracking / IOC creation.

```
file Deception.exe
sha256sum Deception.exe
md5sum Deception.exe
```

File type: PE32+ executable (console) x86-64 (Windows)

SHA-256: 50ecfaefcd8b5b169123618169d9a0696dcbfa81f7991edb93bbb4128d676f8

MD5: 81cf6d116c220670873ac12b37300ff6

Import inspection showed strong indicators of process injection / hollowing: `CreateProcessA`, `ReadProcessMemory`, `WriteProcessMemory`, `ResumeThread`, `WaitForSingleObject`, and `NtQueryInformationProcess`. Strings also referenced a legitimate Windows binary: `C:\\Windows\\System32\\WerFault.exe` — a common “living-off-the-land” target for hollowing.

Finding the Real Payload (embedded shellcode)

Disassembly revealed a function named `ProcessInject` that copies a fixed-size blob from `.rdata` and writes it into the remote process memory:

```
memcpy(stack_buffer, &.rdata[0x4090], 0x3A1);  
WriteProcessMemory(hProcess, remote_addr, stack_buffer, 0x3A1, ...);
```

The blob size (`0x3A1` bytes) and its byte patterns matched position-independent x64 shellcode. Extracting this region gave us the actual “interesting” logic.

Decrypt Routine: SystemFunction032 (RC4)

Inside the embedded shellcode, the sample dynamically resolves `advapi32.dll` and `SystemFunction032` (undocumented RC4 routine). The key trick: both the RC4 key and ciphertext are embedded as raw constants, then wrapped into two `USTRING` structs:

- **Key length:** 9 bytes
- **Ciphertext length:** 27 bytes
- **Deception:** decrypt → `Sleep(2000)` → re-encrypt (flag exists in plaintext only briefly)

From the constants written into stack memory, we reconstruct: `key = 9 bytes` and `ciphertext = 27 bytes`, then RC4-decrypt offline.

Flag Extraction (offline RC4)

Below is the exact solver script used to recover the plaintext flag from the embedded ciphertext:

```
import struct

def rc4(key: bytes, data: bytes) -> bytes:
    S = list(range(256))
    j = 0
    for i in range(256):
        j = (j + S[i] + key[i % len(key)]) & 0xff
        S[i], S[j] = S[j], S[i]
    i = j = 0
    out = bytearray()
    for b in data:
        i = (i + 1) & 0xff
        j = (j + S[i]) & 0xff
        S[i], S[j] = S[j], S[i]
        out.append(b ^ S[(S[i] + S[j]) & 0xff])
    return bytes(out)

le64 = lambda x: struct.pack("<Q", x)
le32 = lambda x: struct.pack("<I", x)

# 27-byte ciphertext built from movabs writes (little-endian), plus final dword of cipher = bytearray()
cipher += le64(0xd04944ca51973913)
cipher += le64(0x74278a61f010894b)
cipher += le64(0xb41e269c52b6f950)
cipher += b"\x00" * 3 # extend to 27 bytes
cipher[23:27] = le32(0xed29e2b4) # overwrite at offset 0x17

# 9-byte key: qword + 1 trailing byte
key = le64(0xff12231287718223) + b"\xee"

print(rc4(key, bytes(cipher)).decode())
```

Running it yields the flag:

IDEH{D3c3pt10n_h1d35_tru7h}

What the Malware Was Trying to “Deceive”

The challenge title matches the behavior: the plaintext flag is intentionally short-lived (decrypt briefly, then re-encrypt), and the outer binary focuses on noisy injection behavior (WerFault hollowing) to distract analysis away from the real secret — a tiny RC4 blob.

IOC / Behavior Summary

- **Target process:** `WerFault.exe`
- **Technique:** Process injection / hollowing (`WriteProcessMemory` + `ResumeThread`)
- **Crypto:** RC4 via `SystemFunction032` (`advapi32`), key length 9
- **Evasion trick:** “decrypt window” limited by `Sleep(2000)`

Final Notes

For real-world defense: flag any binary that spawns a signed Windows process suspended, writes memory into it, then resumes the thread. Additionally, treat calls to `SystemFunction032` as suspicious when paired with high-entropy embedded constants.